AGENTIC SERVER-SIDE AUTHENTICATION VALIDATION

Server-side authentication is the process where a backend server verifies a user's identity and manages their session data, keeping sensitive logic and credentials hidden from the client browser

Server-side authentication validation is the non-negotiable security layer of any application. While the frontend provides a smooth user experience by catching typos and formatting errors, the server is the ultimate gatekeeper. We can never trust client data. In this demo, we are going to use an agentic workflow to incrementally build, verify, and secure our backend authentication pipeline. We will guide the AI to establish strict schema validation, build a secure one-time password (OTP) verification system, and lock down user sessions.

Here is the step-by-step agentic workflow and respective prompts:

1. Context Discovery & Inspection

The Prompt: *"Is there Zod or Joy for server side auth validation? Say yes or no before you start writing an answer."*

> **Purpose:**
> "Before we command the agent to write new features, we need it to understand the current environment. This prompt forces the AI to inspect the existing backend state and give a binary confirmation. It prevents the agent from blindly overwriting code or duplicating libraries."

2. Establishing the Validation Layer

The Prompt: *"Install Zod schema validation library and set it up to perform server side validation during authentication."*

> **Purpose:**
> "Now we build the shield. We are instructing the agent to implement Zod. This ensures that every payload hitting our backend is strictly typed and validated before it ever touches our database, protecting us from injection attacks and malformed data."

3. The Sanity Check

The Prompt: *"Is Zod validation active before we proceed? Yes or no."*

> **Purpose:**
> "In agentic software engineering, trust is good, but verification is better. Before moving on to complex logic, we force a hard stop to verify that the schema validation is actually active. This micro-step prevents compounding errors later in the pipeline."

4. Post-Authentication Actions

The Prompt: *"Install and configure Nodemailer and use it to send welcome emails to new users in the application."*

> **Purpose:**
> "With our initial gate secured, we move to the onboarding flow. Here, we direct the agent to hook up Nodemailer, automating the transactional welcome emails the moment a user successfully clears the validation and database creation steps."

5. Secure Identity Verification & UX

The Prompt: *"Implement email verification for the users in the app. Generate and send a six-digit one-time code... Use random bytes as the code generator... take them to the dashboard and only display a banner... allow them to verify their email address."*

> **Purpose:**
> "This is a massive prompt that handles both backend cryptography and frontend integration. We specifically instruct the agent to use `randomBytes`—ensuring cryptographically secure OTPs rather than predictable math functions. We also define the exact UX: let them in, but gate their capabilities behind a seamless, no-reload verification banner."

6. Iterative Debugging & Enforcing Constraints

The Prompt: *"The generated verification code sometimes fails to be attached to the user record in the database. Also ensure that the verification code is temporary with a timer of let's say 2 minutes."*

> **Purpose:**
> "Agents aren't perfect on the first try. Here, we demonstrate how to course-correct the AI when a database write fails. More importantly, we introduce a critical security constraint: Time-to-Live (TTL). A permanent OTP is a massive vulnerability, so we cap its lifespan at 2 minutes."

7. Database Sanitization

The Prompt: *"When the verification code expires, it should be wiped off the database and the field with a new code when the user requests."*

> **Purpose:**
> "We don't just want the code to be invalid; we want it gone. This prompt directs the agent to practice good data hygiene and secure the app against replay attacks or brute-force cURL exploits by wiping expired tokens directly from the database."

8. Locking Down State

The Prompt: *"Currently when a user logs in nothing is recorded in the session. Set up an active session and verify the session with JWT tokens. Ensure that when a user logs in the session is recorded and verified, and when they log out the session is dead or ended."*

> **Purpose:**
> "For our final step, we secure the user's ongoing state. We instruct the agent to implement JSON Web Tokens (JWT). This establishes a stateless, verified session that explicitly tracks when a user logs in, authenticates their subsequent requests, and securely destroys the token upon logout."